TVVS – Test, Verification, and Validation of Software (M.EIC)
ST – Software Testing (MESW)

Property-Based Testing

Ana Paiva

apaiva@fe.up.pt **www.fe.up.pt/~apaiva**

Learning goals

- Explain what property-based testing is and when it is useful.
- Translate partitions and boundaries into executable properties.
- Use *jqwik* to generate many inputs automatically.
- Understand arbitraries, constraints, edge cases, seeds, and shrinking.
- Recognize common mistakes when designing properties.
- Apply PBT to both pure functions and simple stateful systems.

Equivalence Class Partitioning and Boundary Value Analysis

QUICK RECAP

Running example: passing grade

- A student passes an exam if he/she gets a grade >= 9.5 and fails if he/she gets a grade < 9.5. Grades fall in the range [0.0, 20.0].

```
public class PassingGrade {

    public boolean passed(double grade) {
        if (grade < 0.0 || grade > 20.0)
            throw new IllegalArgumentException();

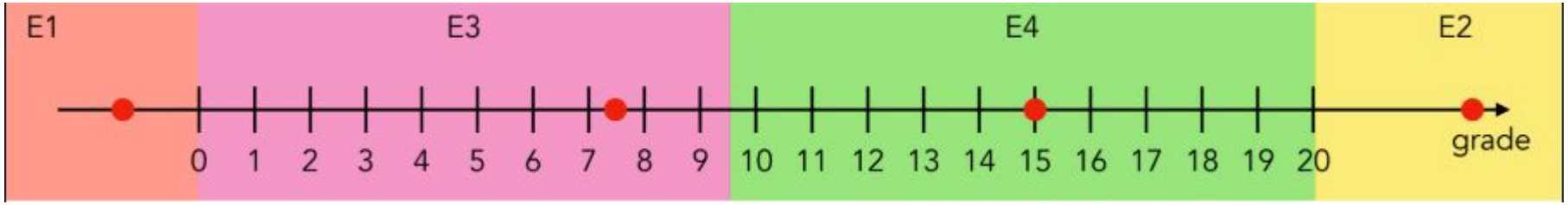
        return grade >= 9.5;
    }
}
```

Equivalence classes

The input domain can be divided into four behavioral regions:

- E1: invalid grade below the minimum] … , 0 [
- E2: invalid grade above the maximum] 20 , … [
- E3: valid failing grade [0 , 9.5 [
- E4: valid passing grade [9.5 , 20]

Traditional testing picks representative examples from each class.



Example inputs: -3.5 (E1); 29 (E2); 7.3 (E3); 15 (E4)

Boundary values

- Boundary Value Analysis focuses on points where behavior changes.
- For the passing grade program, the important boundaries are:
  - minimum valid grade: 0.0
  - pass/fail threshold: 9.5
  - maximum valid grade: 20.0
- Off-points are values just outside or just beside those boundaries.

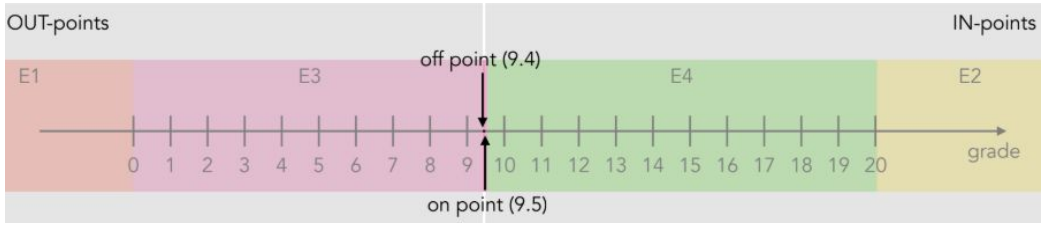
Typical boundary tests: -0.1 · 0.0 · 9.4 · 9.5 · 20.0 · 20.1

Boundary values

If we were to apply boundary value analysis to this program, we would identify the following boundaries:

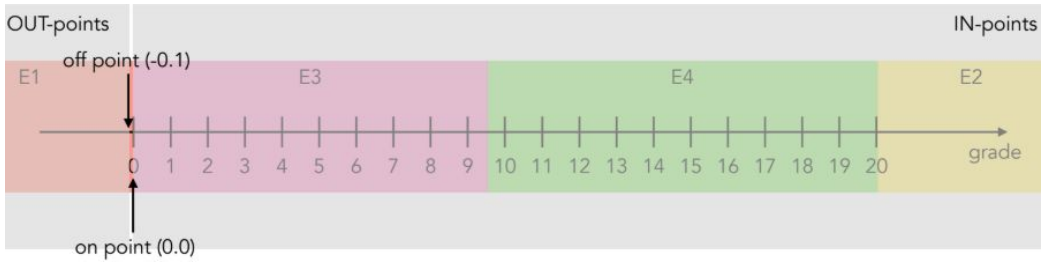
On point at 9.5 and

Off point at 9.4



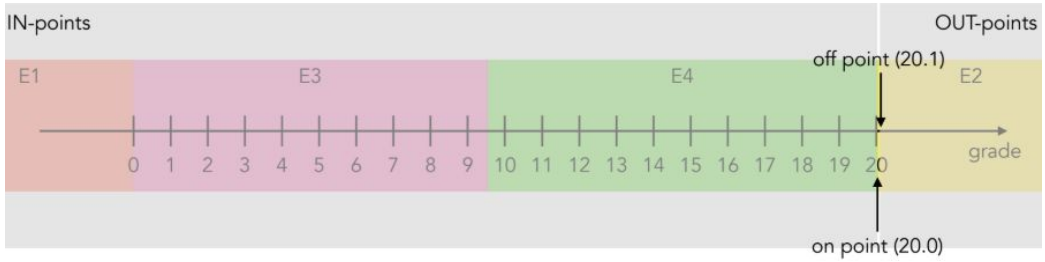
On point at 0.0 and

Off point at -0.1



On point at 20.0 and

Off point at 20.1



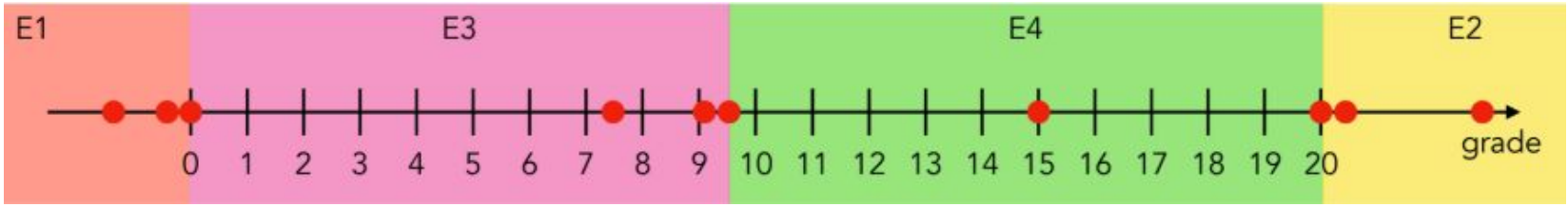
Equivalence Class Partitioning and Boundary Value Analysis

Equivalence class partitioning:

-3.5 (E1); 29 (E2); 7.3 (E3); and 15 (E4)

Boundary Value Analysis

9.5 and 9.4; 0.0 and -0.1; 20.0 and 20.1



We may keep the boundaries and remove redundant elements if testing the same classes

- E1: -3.5; -0.1; -- keep -0.1
- E2: 20.1; 29; -- keep 20.1
- E3: 0.0; 7.3; 9.4; -- keep 0.0 and 9.4
- E4: 9.5; 15; 20.0: -- keep 9.5 and 20.0

Reduce from 10 to 6 test cases

Describe properties; let the framework search for counterexamples

PROPERTY-BASED TESTING

What is Property-Based Testing?

- Property-Based Testing is a testing style where we specify general properties of the program.
- The framework generates many inputs and checks whether the property holds.
- A failure is reported as a concrete counterexample.
- The goal is not to prove correctness.
- The goal is to automatically search for evidence that the property is false.

PBT is falsification-oriented: it tries to break your claim.

jqwik in one slide

- *jqwik* is a property-based testing framework for the JVM.
- A property is written as a method annotated with *@Property*.
- Inputs are annotated with *@ForAll*.
- Constraints restrict the generated values.
- When a generated input breaks the property, *jqwik* reports the input and the random seed.
- When possible, *jqwik* also shrinks the failure to a smaller counterexample.
- PBT (either *jqwiki* or any other tool) cannot prove that a property is correct, but only tries to find examples to refute it!
- jqwik's user guide v1.9.3 can be found in

  https://jqwik.net/docs/current/user-guide.html

Pronounced "jay quick".

Other PBT libs for Java

- **JUnit-Quickcheck**: This is where jqwik has stolen the annotation approach. So the way you write your properties is similar. JUnit-Quickcheck is tightly integrated with JUnit 4 and also supports shrinking.
- **QuickTheories**: QuickTheories is completely test framework independent, which has pros and cons. Besides shrinking, it also has an experimental feature in which value generation is influenced by coverage data.
- **Vavr**: The functional library for Java also comes with a property-based testing module. Currently no shrinking support, though.
- **ScalaCheck**: Definitely a mature property-based testing system with shrinking and all, if you prefer Scala over Java. There's also a book out there about ScalaCheck.
- **test.check for Clojure**: Strongly inspired and influenced by QuickCheck. Since Clojure does not have static types, generators must always be declared explicitly.
- **KotlinTest** also has basic support for PBT. Currently no shrinking yet.
- **Frege**, a Haskell for the JVM, comes with a classical QuickCheck implementation.

Minimal jqwik dependency

```
<!-- Maven -->
<dependency>
    <groupId>net.jqwik</groupId>
    <artifactId>jqwik</artifactId>
    <version>1.9.3</version>
    <scope>test</scope>
</dependency>
```

Use the version required by your project or course environment.

A useful mental model

- A property-based test usually has four parts:
  - Generate data.
  - Optionally constrain or construct the data.
  - Run the system under test.
  - Assert a property that should always hold.
- The hard part is often not writing the code.
- The hard part is discovering a precise, useful property.

Bad property + excellent generator = weak test.

Good properties often come from

- The specification.
- Pre-conditions and post-conditions.
- Invariants that should always hold.
- Equivalence classes and boundaries.
- Metamorphic relations.
- Round-trip behavior.
- Comparing with a simpler oracle.

If you cannot state the property clearly, the test will likely be unclear too.

PBT with jqwik, step-by-step

- 1. For each property we want to express, we create a method and annotate it with *@Property*. These methods look like JUnit tests, but instead of containing a single example, they contain an overall property.
- 2. Properties use randomly generated data. jqwik includes several generators for various types (including *Strings*, *Integers*, *Lists*, *Dates*, and so on). jqwik allows you to define different sets of constraints and restrictions to these parameters: for example, to generate only positive *Integers* or only *Strings* with a length between 5 and 10 characters. The *property* method receives all the required data for that test as parameters.
- 3. The property method calls the method under test and asserts that the method's behavior is correct.
- 4. When the test runs, jqwik generates a large amount of random data (following the characteristics you defined) and calls the test for it, looking for an input that would break the property. If jqwik finds an input that causes your test to fail, it reports that input back to the developer. The developer then has an example input that breaks their program.

Arbitraries

- An **Arbitrary** is a **random generator** of a particular class (or primitive). If you recall discrete mathematics, it is the equivalent of saying: *for a given x*, *where x is a natural number*.
- In source code, we can think of *x* as an object of Arbitrary. Most frameworks provide Arbitrary for several common cases, i.e., numbers, booleans, and collections. You can also define your own Arbitraries.
- There are things that make an Arbitrary more useful than being **merely a random generator**, e.g., statistics and shrinking.

Properties

- To define a property, we just need to add the @Property annotation to our test.

```
public class PassingGradeTest {
 @Property
 public void testPassed(@ForAll grade) {
     assertTrue(passed(grade));
 }
}
```

Constraining Parameters

- Sometimes we want to constrain the generated parameters. For example, the following test:

```
@Property
public void testDivision(@ForAll int number) {
  assertEquals(1, number / number);
}
```

- does not work if the number is zero, so we can use the *@Positive* annotation to constrain the number:

```
@Property
public void testDivision(@ForAll @Positive int number) {
  assertEquals(1, number / number);
}
```

Constraining Parameters

- There are several types of constraints that can be applied:
  - @WithNull(value = 0.1) If we want to generate null values, value is the percentage of null values to generate.
  - @Unique Prevents repeated values in a list.
  - @StringLength(value = 0, min = 0, max = 0) A fixed-size string or between min and max characters.
  - @Chars(value = {}), @CharRange(from = 0, to = 0), @NumericChars, @LowerChars, @UpperChars, @AlphaChars, @WhiteSpace Several ways to constraint character generation.
  - @NotEmpty, @Size(value = 0, min = 0, max = 0) To constraint the size of generated lists.
  - @Positive, @Negative, @IntRange(min = 0, max),
  - @DoubleRange(min = 0.0, max), ... To constraint generated numbers.

Constraining Parameterized Types

- If we want to constrain the generation of contained parameter types, we can annotate the parameter type directly:

```
@Property
public void testListSumPositive(
 @ForAll @NotEmpty List<@Positive Integer> list) {
     assertTrue(sum(list) > 0);
 }
}

private int sum(List<Integer> list) {
 int sum = 0;
 for (int e : list)
     sum += e;
 return sum;
}
```

- ⚠ This property does not hold. Any clue?

Arbitrary

- If the default generators are not enough, we can use the *@Provide* annotation and *Arbitrary* class to create new generators:

```
@Property
public void testDivision(@ForAll("notZero") int number) {
 assertEquals(1, number / number);
}
@Provide
Arbitrary<Integer> notZero() {
 return Arbitraries.integers().filter(n -> n != 0);
}
```

- With Arbitraries we can generate integers(), strings(), ... which we can then restrict using functions like filter(f), map(f), greaterOrEqual(n), alpha(), numeric(), ofLength(n), ...

Combining Arbitraries

- We can even combine arbitraries using the *combine* and *as* methods, sprinkled with some Java lambda magic:

```
@Property void testWithPlates(@ForAll("carPlates") String plate) {
 // do some testing using plate
}

@Provide
Arbitrary<String> carPlates() {
   return Combinators.combine(
 Arbitraries.strings().alpha().ofLength(2),
 Arbitraries.strings().numeric().ofLength(2),
 Arbitraries.strings().numeric().ofLength(2)
   ).as((s1, s2, s3) -> s1.toUpperCase() + "-" + s2 + "-" + s3);
}
```

Properties for the passing grade program (running example)

Passing grade property:

Every valid grade at or above the pass threshold must return true.

Invalid grade property:

Every grade below the minimum or above the maximum must cause an exception.

Failing grade property:

Every valid grade below the pass threshold must return false.

These properties correspond directly to the equivalence classes.

Property 1: passing grades

```
class PassingGradeTest {

    private final PassingGrade pg = new PassingGrade();

    @Property
    void validGradesAtOrAboveThresholdShouldPass(
        @ForAll @DoubleRange(min = 9.5, max = 20.0)
        double grade
    ) {
        assertTrue(pg.passed(grade));
    }
}
```

Property 2: invalid grades

```
class PassingGradeTest {
  private final PassingGrade pg = new PassingGrade();

  @Property
  void invalidGradesShouldThrow(@ForAll("invalidGrades") double grade) {
      assertThrows(IllegalArgumentException.class, () -> pg.passed(grade));
  }

  @Provide
  Arbitrary<Double> invalidGrades() {
      Arbitrary<Double> tooSmall =
          Arbitraries.doubles().lessThan(0.0);

      Arbitrary<Double> tooLarge =
          Arbitraries.doubles().greaterThan(20.0);

      return Arbitraries.oneOf(tooSmall, tooLarge);
  }
}
```

Property 3: failing grades

```
class PassingGradeProperties {

    private final PassingGrade pg = new PassingGrade();

    @Property
    void validGradesBelowThresholdShouldFail(
        @ForAll @DoubleRange(min = 0.0, max = 9.5, maxIncluded = false)
        double grade
    ) {
        assertFalse(pg.passed(grade));
    }
}
```

Common Property Patterns

Pattern	Question to ask	Example
Invariant	What must always be true for all valid inputs?	Dividing zero by any non-zero number always gives zero.
Round-trip	Can we go there and back safely?	parse(format(date)) == date
Idempotence	Does repeating the operation change anything?	normalise(normalise(text)) == normalise(text)
Metamorphic relation	If input changes in a known way, how should output change?	Adding an item increases the cart total by its price.

Shrinking

- When jqwik finds a failing input, it tries to simplify it.
- This process is called shrinking.
- The final counterexample is usually smaller and easier to understand.
- Example:
  - A property may fail for a list with 200 elements.
  - Shrinking may reduce it to a list with 2 elements.
- Shrinking makes debugging much faster.

Random testing finds the bug. Shrinking explains the bug.

Understanding jqwik output

```
|--------------------jqwik--------------------
tries = 1000
checks = 1000
generation-mode = RANDOMIZED
after-failure = PREVIOUS_SEED
seed = 52969275234446902 3
```

tries: number of generated attempts.

checks: number of non-rejected attempts.

seed: value used to reproduce the generated data.

From single inputs to sequences of operations

STATEFUL PROPERTY-BASED TESTING

The stack example

- For data structures, one input is often not enough.
- Bugs may appear only after a sequence of operations.
- For a stack of strings, relevant operations include:
  - push
  - pop
  - clear
  - isEmpty
  - size
- Stateful PBT generates operation sequences and checks invariants after each step.

Stack invariants

- After creating a stack, it should be empty.
- After pushing one element, the stack should not be empty.
- After pushing an element and then popping, the popped element should be the same.
- After clearing, the stack should be empty.
- The size should never be negative.
- If the stack is empty, pop should either fail in a specified way or return a specified empty result.

The important part: define the expected behavior before generating sequences.

Example: simple stack property

```
@Property
void pushThenPopReturnsSameValue(@ForAll String value) {

    MyStringStack stack = new MyStringStack();

    stack.push(value);

    assertFalse(stack.isEmpty());
    assertEquals(value, stack.pop());
    assertTrue(stack.isEmpty());

}
```

From operation coverage to behavioral exploration

- Model-based testing asks us to cover transitions.
- Property-Based Testing can generate many paths through the model.
- For the stack, generated scenarios may include:
  - push, push, pop
  - push, clear, push, pop
  - clear, clear, push
  - push, pop, pop
- The test oracle still matters: the model tells us what should happen.

PBT is powerful, but easy to misuse

COMMON ISSUES

Issue 1: properties that are too weak

- A property can pass while checking almost nothing.
- Example of a weak property:
  - The result is not null.
- Sometimes that is useful, but often it is not enough.
- A stronger property explains the expected relationship between input and output.

Ask: if this property passes, what have I actually learned?

Issue 2: over-filtering

Requiring **jqwik** to generate data that is very expensive or even impossible. The method below tries to generate an array by randomly producing integers and filtering them until they satisfy all constraints. It asks jqwik to randomly find numbers and then discards the ones that do not match the constraints.

```
@Provide
private Arbitrary<int[]> uniqueMultiplesOf2_3_5_15_badExample() {
     return Arbitraries.integers()
             .between(1, 10_000)
             .filter(number -> number % 2 == 0)
             .filter(number -> number % 3 == 0)
             .filter(number -> number % 5 == 0)
             .filter(number -> number % 15 == 0)
             .array(int[].class)
             .ofSize(100)
             .uniqueElements();
 }
```

Filtering can reject too many generated candidates.

Rejected candidates reduce useful test coverage.

Prefer building valid data directly when possible.

Issue 3: unfair data distribution

Uniform random generation does not guarantee meaningful scenario coverage.

Triangle example: Generating three arbitrary side lengths often produces invalid triangles. The valid triangle property may be barely exercised.

```
public class Triangle {
  public boolean isNotTriangle(int a, int b, int c) {
    return a >= (b + c) || c >= (b + a) || b >= (a + c);
  }
}

@Property
public void triangleTest(
  @ForAll @IntRange(max = 100) int a,
  @ForAll @IntRange(max = 100) int b,
  @ForAll @IntRange(max = 100) int c) {
    // ... test code ...
}
```

Better approach:

Create one generator for valid triangles.
Create another generator for invalid triangles.
Test the properties separately.

Issue 4: floating-point values

- Floating-point numbers can introduce surprising edge cases and boundaries are perfect place for bugs. jqwik will do its best to generate boundary values, for example, 1) the closest possible number to 1f; or 2) the smallest possible float. The default configuration for jqwik is to mix edge cases with random data points.
- Consider whether your generator should include or exclude:
  - NaN
  - positive and negative infinity
  - negative zero
  - very small values close to zero
- For business domains such as grades or money, decimal types may be more appropriate.

The generator should match the domain, not just the Java type.

PBT and “other” techniques

- Property-Based Testing does not replace other testing techniques.
- It complements equivalence classes and boundary values.
- A good workflow:
  - Use examples for known cases and regression bugs.
  - Use ECP and BVA to understand the input space.
  - Turn those insights into properties.
  - Use PBT to search broadly for counterexamples.
  - Use structural testing to inspect code coverage and missing paths.

A practical recipe

1. Start from the specification.
2. Identify valid and invalid input domains.
3. Write one property per behavior.
4. Design generators that produce meaningful data.
5. Include boundaries and edge cases.
6. Check jqwik output, especially rejected cases.
7. When a property fails, keep the seed and add a regression example if useful.

Summary

- Property-Based Testing expresses behaviors as general properties.
- The framework generates many inputs and searches for counterexamples.
- Generators and constraints are part of the test design.
- Shrinking turns large failures into smaller, more useful examples.
- PBT is especially useful for broad input spaces, transformations, parsers, data structures, and stateful systems.
- The hardest skill is writing properties that are precise, meaningful, and testable.

Suggested homework

- Exercise A: write the three PassingGrade properties in jqwik.
- Exercise B: deliberately introduce a bug and observe jqwik's counterexample.
- Exercise C: write properties for a Stack implementation.
- Exercise D: design separate generators for valid and invalid triangles.
- Discussion: which properties were easy to express, and which ones required creativity?

References

- Maurício Aniche, Effective Software Testing, Manning, 2022.
- Koen Claessen and John Hughes, QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs, 2000.
- jqwik User Guide: https://jqwik.net/docs/current/user-guide.html
- Bertrand Meyer, Object-Oriented Software Construction, 2nd edition, 1997.
- Eric Evans, Domain-Driven Design, Addison-Wesley, 2004.
- John Hughes, How to Specify It! A Guide to Writing Properties of Pure Functions, 2020.